

# STM – DOCKER E KUBERNETES

Instrutoria Interna

Introdução à Containers e Docker

Evolução da Infraestrutura, Arquitetura do Docker e Primeiros Passos

Wescley de Macêdo Sousa  
[wescleysousa@stm.jus.br](mailto:wescleysousa@stm.jus.br)



**STM – SUPERIOR TRIBUNAL  
MILITAR**

**DITIN/CDESC**

Criado em 19/09/2026

**INTRODUÇÃO À CONTAINERS E DOCKER**

## **INTRODUÇÃO À CONTAINERS E DOCKER**



**INTRODUÇÃO À CONTAINERS E DOCKER**

**SUMÁRIO**

|                                                                      |           |
|----------------------------------------------------------------------|-----------|
| <b>MÓDULO 1: EVOLUÇÃO DA INFRAESTRUTURA E CONCEITOS FUNDAMENTAIS</b> | <b>3</b>  |
| 1.1 – APLICAÇÕES TRADICIONAIS E SEUS DESAFIOS (ERA BARE METAL)       | 3         |
| 1.2 – VIRTUALIZAÇÃO TRADICIONAL (MÁQUINAS VIRTUAIS)                  | 4         |
| 1.3 – MÁQUINAS VIRTUAIS VERSUS CONTAINERS                            | 5         |
| 1.4 – CONCEITO DE CONTAINER E PRIMITIVAS DO KERNEL LINUX             | 6         |
| 1.5 IMAGEM VERSUS CONTAINER: A RELAÇÃO FUNDAMENTAL                   | 8         |
| 1.6 BENEFÍCIOS E LIMITAÇÕES DA CONTEINERIZAÇÃO                       | 9         |
| <b>MÓDULO 2: A PLATAFORMA DOCKER E SUA ARQUITETURA</b>               | <b>10</b> |
| 2.1 – O QUE É O DOCKER?                                              | 10        |
| 2.2 – ARQUITETURA INTERNA DO DOCKER                                  | 10        |
| 2.3 – DOCKER ENGINE E DOCKER CLI                                     | 12        |
| 2.4 – REGISTRIES E DOCKER HUB                                        | 12        |
| <b>MÓDULO 3: COMANDOS ESSENCIAIS E CICLO DE VIDA DE CONTAINERS</b>   | <b>13</b> |
| 3.1 – O CICLO DE VIDA DE UM CONTAINER                                | 13        |
| 3.2 – EXPLORAÇÃO DETALHADA DOS COMANDOS DA CLI                       | 14        |
| <b>MÓDULO 4: LABORATÓRIO E DESAFIO PRÁTICO GUIADO</b>                | <b>22</b> |
| <b>RESUMO DOS COMANDOS EXECUTADOS NO LABORATÓRIO</b>                 | <b>28</b> |
| <b>REFERÊNCIAS</b>                                                   | <b>29</b> |



INTRODUÇÃO À CONTAINERS E DOCKER

## MÓDULO 1: EVOLUÇÃO DA INFRAESTRUTURA E CONCEITOS FUNDAMENTAIS

### 1.1 – APLICAÇÕES TRADICIONAIS E SEUS DESAFIOS (ERA BARE METAL)

Nos primórdios da computação corporativa, o modelo dominante de implantação de software era a execução direta sobre o hardware físico, conhecido como **Bare Metal**. Nesse paradigma, um sistema operacional (como Linux Red Hat, Debian ou Windows Server) era instalado diretamente sobre os componentes físicos do servidor (CPU, memória RAM, discos e placas de rede), e sobre esse sistema operacional instalavam-se os bancos de dados, servidores web e linguagens de programação necessários para a aplicação.



Embora esse modelo oferecesse acesso direto aos recursos de hardware com o mínimo de latência, ele impôs severos desafios operacionais e financeiros às organizações:

1. **Subutilização massiva de hardware:** Para garantir que picos de acesso não derrubassem sistemas críticos, os servidores eram superdimensionados. Como resultado, a média histórica de uso de CPU e memória RAM em servidores Bare Metal mantinha-se entre **5% e 15%** da capacidade total.
2. **Conflito de dependências ("Dependency Hell"):** Tentar executar duas aplicações no mesmo servidor físico frequentemente gerava conflitos inconciliáveis. Se a *Aplicação A* exigia a versão 1.0 de uma biblioteca e a *Aplicação B* exigia a versão 2.0 da mesma biblioteca, a convivência no mesmo sistema de arquivos tornou-se quase impossível.
3. **Escalabilidade lenta e onerosa:** Adicionar capacidade computacional exigia o processo físico de aquisição (*procurement*), montagem em rack, cabeamento e



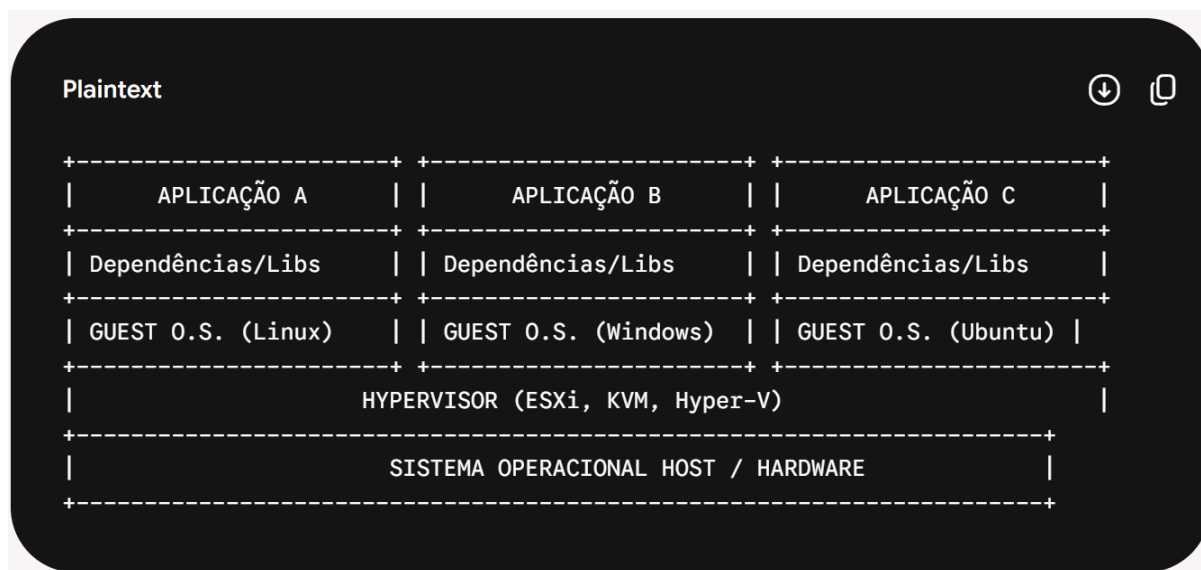
## INTRODUÇÃO À CONTAINERS E DOCKER

configuração do sistema operacional — um processo que levava semanas ou meses.

4. **Falta de reprodutibilidade ("Na minha máquina funciona"):** O ambiente em que o desenvolvedor construía o software (com suas variáveis de ambiente, versões de pacotes e patches específicos) diferia drasticamente do ambiente de homologação e produção. O resultado era o surgimento constante de *bugs* imprevisíveis no momento do *deploy*.

### 1.2 – VIRTUALIZAÇÃO TRADICIONAL (MÁQUINAS VIRTUAIS)

Na década de 2000, a ascensão dos **Hypervisors** (Softwares de Virtualização) revolucionou a infraestrutura de TI. A virtualização introduziu uma camada de abstração entre o hardware físico e o sistema operacional, permitindo fatiar um único servidor físico em múltiplas **Máquinas Virtuais (VMs)** independentes.



Existem dois tipos primários de Hypervisors:

- **Hypervisor Tipo 1 (Bare Metal):** Executa diretamente sobre o hardware físico (ex: VMware ESXi, Proxmox VE, KVM).
- **Hypervisor Tipo 2 (Hosted):** Executa como uma aplicação sobre um sistema operacional hospedeiro (ex: VirtualBox, VMware Workstation).



## INTRODUÇÃO À CONTAINERS E DOCKER

Cada Máquina Virtual é um computador completo sintetizado por software. Ela possui sua própria BIOS/UEFI, drivers virtuais, alocação rígida de memória e, principalmente, um **Guest Operating System (Sistema Operacional Convidado)** completo rodando seu próprio Kernel.

### Desafios Mantidos pela Virtualização Tradicional:

Apesar do grande ganho em isolamento e consolidação de servidores, as VMs trouxeram novas ineficiências:

- **Alto consumo de recursos (Overhead):** Cada VM consome gigabytes de RAM e centenas de megabytes de disco apenas para manter o seu *Guest OS* em execução, antes mesmo de carregar a aplicação final.
- **Tempo de inicialização lento:** Iniciar uma VM exige o processo completo de *boot* do sistema operacional (POST, Kernel init, systemd/services), levando de dezenas de segundos a vários minutos.
- **Alocação estática e desperdício:** Se uma VM é configurada com 16 GB de RAM, esse montante é reservado do host físico, mesmo que a aplicação esteja utilizando apenas 500 MB no momento.

### 1.3 – MÁQUINAS VIRTUAIS VERSUS CONTAINERS

A containerização não busca substituir a virtualização em todos os cenários, mas propõe uma abordagem radicalmente diferente para o isolamento de aplicações: em vez de **virtualizar o hardware**, o container **virtualiza o Sistema Operacional**.

| MÁQUINA VIRTUAL                       | CONTAINER                        |
|---------------------------------------|----------------------------------|
| Aplicação A                           | Aplicação A                      |
| Libs / Dependências                   | Libs / Dependências              |
| GUEST O.S. (Kernel + Binários + Libs) | Visão Isolada do SO (User Space) |
| HYPERVISOR                            | ENGINE DE CONTAINER              |
| SISTEMA OPERACIONAL HOST              | SISTEMA OPERACIONAL HOST         |
| HARDWARE FÍSICO                       | HARDWARE FÍSICO                  |



## INTRODUÇÃO À CONTAINERS E DOCKER

Tanto os containers quanto as VMs fornecem ambientes de execução isolados, mas utilizam arquiteturas fundamentalmente distintas:

| Característica                 | Máquina Virtual (VM)                  | Container (Docker)                                |
|--------------------------------|---------------------------------------|---------------------------------------------------|
| <b>Camada de Abstração</b>     | Virtualização do Hardware físico.     | Virtualização do Kernel do Sistema Operacional.   |
| <b>Sistema Operacional</b>     | Cada VM executa um Guest OS completo. | Compartilha o Kernel do Sistema Operacional Host. |
| <b>Tamanho de Imagem/Disco</b> | Grande (Geralmente 10 GB a 50 GB).    | Leve (Geralmente 5 MB a 500 MB).                  |
| <b>Tempo de Boot</b>           | Minutos ou dezenas de segundos.       | Milissegundos a poucos segundos.                  |
| <b>Consumo de Memória/CPU</b>  | Reservado e rígido (Alto overhead).   | Dinâmico e sob demanda (Overhead mínimo).         |
| <b>Isolamento</b>              | Forte (Nível de Hardware/Hypervisor). | Moderado/Forte (Nível de Processo no Kernel).     |
| <b>Densidade por Servidor</b>  | Dezenas de VMs por servidor físico.   | Centenas ou milhares de Containers por servidor.  |

### 1.4 – CONCEITO DE CONTAINER E PRIMITIVAS DO KERNEL LINUX

Um **Container** nada mais é do que um **processo Linux isolado**, executando diretamente sobre o Kernel do host, mas restrito a uma visão customizada do sistema. Para que um container pareça um "sistema operacional completo" para a aplicação dentro dele, o Docker faz uso de recursos nativos avançados do Kernel do Linux:



## INTRODUÇÃO À CONTAINERS E DOCKER



### 1. Linux Namespaces (Isolamento de Visão)

Os *Namespaces* determinam o que um processo pode **ver** dentro do sistema. Cada container recebe seu próprio conjunto de namespaces:

- **PID Namespace (Process ID):** Garante isolamento nos IDs de processos. A aplicação dentro do container enxerga a si mesma como o processo principal (PID 1), embora no host ela seja apenas mais um PID comum entre milhares.
- **NET Namespace (Network):** Fornece ao container sua própria pilha de rede isolada, incluindo interfaces virtuais (eth0), tabela de roteamento, regras de firewall e portas de escuta.
- **MNT Namespace (Mount):** Isola os pontos de montagem do sistema de arquivos. O container enxerga apenas o seu próprio diretório raiz (/), totalmente desacoplado do diretório raiz do host.
- **IPC Namespace (Inter-Process Communication):** Impede que processos de containers diferentes compartilhem memória compartilhada ou filas de mensagens sem permissão explícita.
- **UTS Namespace (UNIX Timesharing System):** Permite que cada container defina seu próprio *Hostname* e nome de domínio independente.
- **USER Namespace:** Permite mapear usuários de dentro do container para usuários diferentes no host. Um processo executando como root dentro do container pode estar mapeado para um usuário não-privilegiado no host.

### 2. Control Groups - cgroups (Limitação de Recursos)

Enquanto os *Namespaces* isolam o que o processo **vê**, os *Control Groups (cgroups)* controlam o quanto de recurso o processo pode **usar**.

- Permitem definir limites estritos para uso de **CPU, Memória RAM, I/O de Disco e Banda de Rede**.





## **INTRODUÇÃO À CONTAINERS E DOCKER**

- A imagem é um **template estático e imutável** que contém todo o código da aplicação, bibliotecas, variáveis de ambiente, binários e arquivos de configuração necessários para a execução.
- Imagens são compostas por **camadas empilhadas (Layers)** usando o **Union File System (UnionFS / OverlayFS)**. Cada linha de instrução em um Dockerfile cria uma camada somente leitura.
- Camadas idênticas são reutilizadas entre imagens diferentes hospedadas na mesma máquina, economizando espaço em disco e acelerando downloads.

### **Anatomia do Container (Read-Write / Instância Executável)**

- Quando executamos uma imagem com o comando `docker run`, o Docker adiciona uma **camada gravável fina (Container/Writable Layer)** no topo da pilha de camadas da imagem.
- Todas as alterações feitas pelo container em tempo de execução (criação de arquivos de log, gravação temporária, alteração de configurações) ocorrem **exclusivamente** nessa camada gravável superior.
- Se o container for removido, essa camada gravável é destruída permanentemente. A imagem original permanece **intacta e inalterada**.

## **1.6 BENEFÍCIOS E LIMITAÇÕES DA CONTEINERIZAÇÃO**

### **Benefícios Principais:**

1. **Portabilidade Universal:** Se a imagem roda na máquina do desenvolvedor, rodará de forma idêntica em homologação, em servidores locais ou em nuvens públicas (AWS, GCP, Azure).
2. **Eficiência e Densidade Computacional:** Devido à ausência de um Guest OS por container, é possível rodar dezenas de containers no espaço em que rodaria apenas uma VM.
3. **Inicialização Instantânea:** Por ser apenas a inicialização de um processo no Kernel, um container inicia em fração de segundo.
4. **Facilidade em CI/CD:** Pipelines de Integração e Entrega Contínuas se tornam extremamente simples: o artefato compilado é a própria imagem Docker pronta para implantação.

### **Limitações e Desafios:**

1. **Dependência do Kernel do Host:** Um container Linux só pode ser executado em um host que possua Kernel Linux. Não é possível rodar um container nativamente



## **INTRODUÇÃO À CONTAINERS E DOCKER**

compilado para Windows Kernel sobre um Kernel Linux sem camadas de tradução ou VMs intermediárias.

2. **Isolamento de Segurança Relativo:** Como o Kernel é compartilhado entre todos os containers do host, uma vulnerabilidade crítica de execução remota no Kernel do Host pode afetar todos os containers rodando na máquina.
3. **Persistência de Dados Não-Nativa:** Como o sistema de arquivos do container é efêmero (destruído com a remoção do container), aplicações com estado (*stateful*), como bancos de dados, exigem a configuração explícita de **Volumes** ou montagens externas do host.

## **MÓDULO 2: A PLATAFORMA DOCKER E SUA ARQUITETURA**

### **2.1 – O QUE É O DOCKER?**

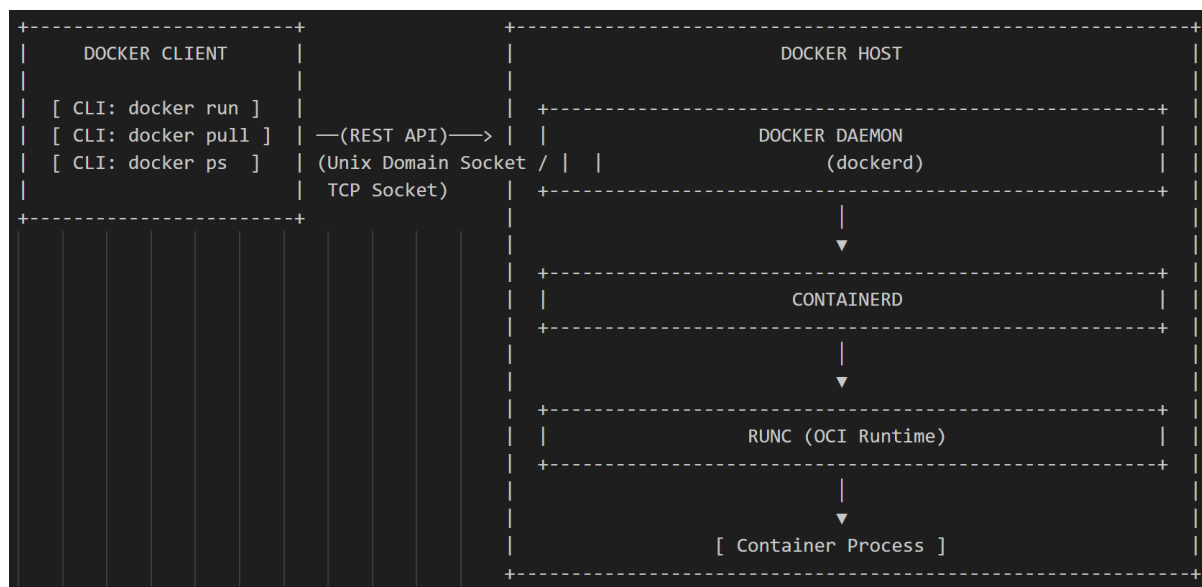
Lançado como projeto Open Source por Solomon Hykes em 2013 (pela empresa DotCloud), o **Docker** popularizou e padronizou o uso de containers no mercado global. O Docker não inventou os containers — o Linux já possuía tecnologias como LXC (Linux Containers), chroot e FreeBSD Jails há anos. A grande revolução do Docker foi criar uma **plataforma intuitiva, padronizada e unificada** para criar, empacotar, distribuir e executar esses containers com comandos simples.

### **2.2 – ARQUITETURA INTERNA DO DOCKER**

O Docker adota uma **arquitetura Cliente-Servidor (Client-Server)** desacoplada. O cliente envia ordens e o servidor realiza todo o trabalho pesado de construção, execução e gerenciamento dos recursos.



## INTRODUÇÃO À CONTAINERS E DOCKER



### Componentes Principais da Arquitetura:

1. **Docker Client (docker CLI):** A ferramenta de linha de comando que os usuários interagem diretamente. O cliente não cria nem executa os containers; ele apenas converte suas entradas em chamadas REST e as envia para o Docker Daemon.
2. **Docker Daemon (dockerd):** O serviço principal de plano de fundo (*background service*) que roda no sistema host. Ele escuta as requisições da API do Docker, gerencia objetos do ecossistema (Imagens, Containers, Redes, Volumes) e coordena a execução.
3. **Comunicação Cliente-Daemon (REST API):** Por padrão, o cliente e o daemon se comunicam localmente via **Unix Domain Socket** localizado no caminho `/var/run/docker.sock`. Também é possível configurar a comunicação remota utilizando instâncias expostas sobre conexões HTTP/HTTPS protegidas por TLS.
4. **containerd:** Um runtime de container de alto nível (*high-level container runtime*) que gerencia o ciclo de vida completo dos containers: download de imagens, gerenciamento de armazenamento, supervisão da execução e métricas de rede.
5. **runc:** Um runtime de baixo nível (*low-level container runtime*) leve e de propósito único que segue estritamente a especificação da **OCI (Open Container Initiative)**. Seu único papel é interagir diretamente com as chamadas de sistema do Kernel do Linux (creating namespaces, cgroups) para dar vida ao processo do container.



## INTRODUÇÃO À CONTAINERS E DOCKER

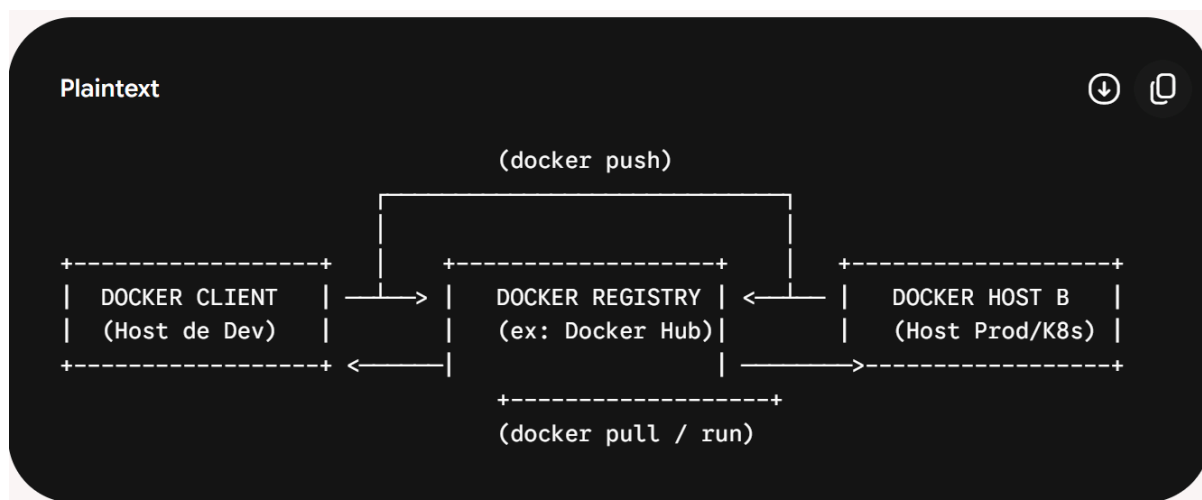
### 2.3 – DOCKER ENGINE E DOCKER CLI

**Docker Engine:** É o conjunto completo de software composto pelo Docker Daemon, pela API REST e pelos runtimes (containerd e runc). É o motor de execução instalado no servidor.

**Docker CLI:** É a interface de linha de comando interativa. Opcionalmente, pode ser executada em uma máquina remota enviando ordens para o Docker Engine de um servidor em nuvem.

### 2.4 – REGISTRIES E DOCKER HUB

Um **Registry** é um serviço de armazenamento centralizado responsável por guardar, versionar e distribuir **Imagens Docker**.



**Docker Hub:** É o *registry* público oficial fornecido pela Docker Inc. ([hub.docker.com](https://hub.docker.com)). Contém centenas de milhares de imagens prontas, incluindo **Imagens Oficiais** mantidas por organizações de prestígio (ex: nginx, python, postgres, ubuntu, alpine).

**Anatomia do Nome de uma Imagem:** A nomenclatura completa de uma imagem segue a estrutura: `registry_url/usuario_ou_org/nome_da_imagem:tag`

- Exemplo Completo: `docker.io/library/nginx:latest`
- Quando omitimos o servidor, o Docker assume automaticamente o `docker.io` (Docker Hub).
- Quando omitimos a organização em imagens oficiais, o Docker assume `library`.



## INTRODUÇÃO À CONTAINERS E DOCKER

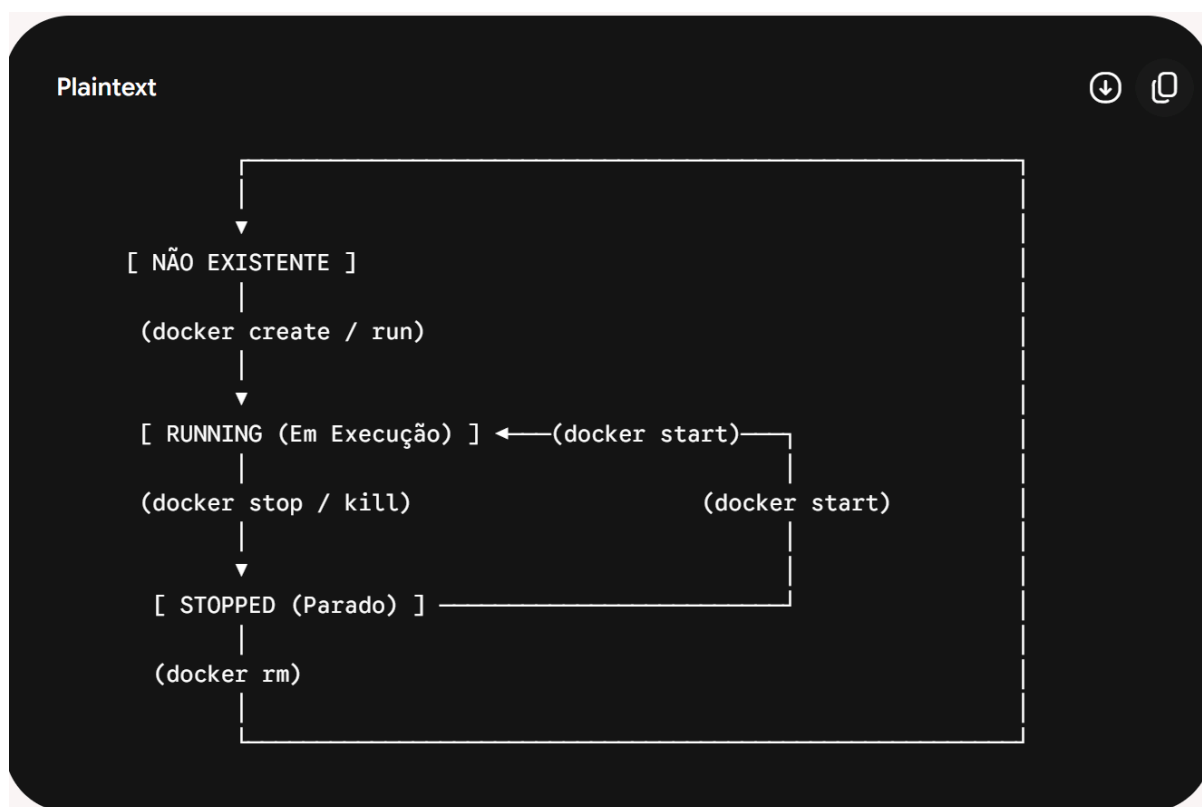
- Quando omitimos a **Tag** (versão), o Docker assume por padrão a tag **:latest**.

**Registries Privados e Corporativos:** Em ambientes corporativos, código proprietário não deve ser publicado no Docker Hub público. Utilizam-se soluções como AWS ECR (Elastic Container Registry), Azure ACR, Google GAR, Harbor ou Nexus Repository.

## MÓDULO 3: COMANDOS ESSENCIAIS E CICLO DE VIDA DE CONTAINERS

### 3.1 – O CICLO DE VIDA DE UM CONTAINER

Um container passa por estados bem definidos durante a sua existência no sistema:



**Created (Criado):** O container teve sua estrutura e camada de escrita alocada, mas o processo principal não foi iniciado.



## INTRODUÇÃO À CONTAINERS E DOCKER

**Running (Em Execução):** O processo principal do container (**PID 1**) está ativo e consumindo CPU/RAM.

**Paused (Pausado):** Os processos do container foram congelados temporariamente via chamadas de sistema *cgroup freezer*.

**Stopped / Exited (Parado):** O processo principal encerrou a execução ou foi finalizado via sinal de término (**SIGTERM** / **SIGKILL**).

**Destroyed / Removed (Removido):** O container e sua camada de escrita foram apagados permanentemente do sistema hospedeiro.

### 3.2 – EXPLORAÇÃO DETALHADA DOS COMANDOS DA CLI

Nesta seção, analisaremos minuciosamente a sintaxe, o comportamento e os parâmetros dos comandos essenciais da CLI do Docker.

#### 1. **docker version**

- **Descrição:** Exibe informações detalhadas sobre as versões do Docker Client e do Docker Engine em execução, incluindo a versão da API REST, a revisão do Git e o sistema operacional/arquitetura.
- **Utilidade:** Validação básica do ambiente e verificação da conectividade entre a CLI e o Daemon.

```
Bash
docker version
```

Exemplo de Saída Esperada:



## INTRODUÇÃO À CONTAINERS E DOCKER

### Plaintext



```
Client: Docker Engine - Community
Version:      24.0.7
API version:  1.43
Go version:   go1.20.10
Git commit:   af2c3f0
Built:        Thu Oct 26 09:07:41 2023
OS/Arch:      linux/amd64

Server: Docker Engine - Community
Engine:
Version:      24.0.7
API version:  1.43 (minimum version 1.12)
Go version:   go1.20.10
Git commit:   311b9ff
Built:        Thu Oct 26 09:07:41 2023
OS/Arch:      linux/amd64
Experimental: false
```

## 2. docker info

- **Descrição:** Exibe estatísticas abrangentes sobre o estado atual do Docker Daemon e da máquina hospedeira.
- **Informações Reveladas:** Quantidade de containers (rodando, parados, pausados), número de imagens baixadas, driver de armazenamento em uso (**OverlayFS2**), diretório raiz do Docker (**/var/lib/docker**), limites de CPU/memória e hostname do host.

### Bash



```
docker info
```

## 3. docker pull



## INTRODUÇÃO À CONTAINERS E DOCKER

- **Descrição:** Faz o download de uma imagem de um *Registry* (como o Docker Hub) para o repositório local do Docker Host sem executá-la.
- **Sintaxe:** `docker pull <nome_da_imagem>:<tag>`

Bash



```
docker pull nginx:alpine
```

**i** Nota de Engenharia: A variante `:alpine` do Nginx é baseada na distribuição Linux Alpine, conhecida por ser extremamente leve (aproximadamente 10 MB a 20 MB), em comparação com a versão padrão baseada em Debian (aproximadamente 140 MB).

### 4. docker images (ou docker image ls)

- **Descrição:** Lista todas as imagens armazenadas no repositório local da sua máquina.
- **Colunas de Saída:**
  - REPOSITORY: O nome da imagem/projeto.
  - TAG: O identificador da versão (ex: alpine, 1.25, latest).
  - IMAGE ID: O hash SHA-256 truncado que identifica univocamente a imagem.
  - CREATED: Há quanto tempo a imagem foi compilada.
  - SIZE: O espaço total que a imagem ocupa no disco.

Bash



```
docker images
```

### 5. docker run



## INTRODUÇÃO À CONTAINERS E DOCKER

- **Descrição:** É o comando mais importante e complexo do Docker. Ele é a junção do processo de criação de container (`docker create`) com a sua inicialização (`docker start`).
- **Sintaxe Geral:** `docker run [FLAGS] <NOME_DA_IMAGEM> [COMANDO] [ARGUMENTOS]`

### Flags Principais do `docker run`:

| Flag                | Nome Curto / Longo     | Descrição e Função                                                                                                      |
|---------------------|------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>-d</code>     | <code>--detach</code>  | Executa o container em segundo plano ( <i>Detached mode</i> ) e libera o terminal.                                      |
| <code>-it</code>    | <code>-i --tty</code>  | Conecta o terminal interativo. <code>-i</code> mantém o <code>STDIN</code> aberto; <code>-t</code> aloca um pseudo-TTY. |
| <code>-p</code>     | <code>--publish</code> | Mapeia/expõe portas no formato<br><Porta_do_Host>:<br><Porta_do_Container><br>.                                         |
| <code>--name</code> | <code>--name</code>    | Atribui um nome customizado ao container. Se omitido, o Docker gera um nome aleatório.                                  |
| <code>-v</code>     | <code>--volume</code>  | Monta um volume ou diretório do host dentro do container para persistência.                                             |
| <code>--e</code>    | <code>--env</code>     | Injeta uma variável de ambiente dentro do container.                                                                    |
| <code>--rm</code>   | <code>--rm</code>      | Remove automaticamente o container e sua camada de escrita assim que ele parar.                                         |



## INTRODUÇÃO À CONTAINERS E DOCKER

Exemplo de Execução Simples:

Bash



```
docker run nginx
```

⚠ **Atenção:** Executar o comando sem a flag `-d` fará com que o container rode em primeiro plano (Foreground). Os logs da aplicação ocuparão a sua tela e, se você pressionar `Ctrl + C`, o processo principal receberá um sinal para encerrar e o container será parado.

### 6. docker ps

- **Descrição:** Lista os containers que estão atualmente em estado de execução (Running).
- **Colunas de Saída:** CONTAINER ID, IMAGE, COMMAND (o processo executado), CREATED, STATUS, PORTS (mapeamentos de rede) e NAMES.

Bash



```
docker ps
```

### 7. docker ps -a

- **Descrição:** A flag `-a` (ou `--all`) estende a listagem para exibir **todos os containers** no host, independentemente do status (incluindo containers parados, finalizados com erro ou recém-criados).



## INTRODUÇÃO À CONTAINERS E DOCKER

Bash



```
docker ps -a
```

Exemplo de Saída:

Plaintext



| CONTAINER ID | IMAGE        | COMMAND                 | CREATED        | STATUS       |
|--------------|--------------|-------------------------|----------------|--------------|
| a1b2c3d4e5f6 | nginx:alpine | "/docker-entrypoint..." | 2 minutes ago  | Up 2 minutes |
| f9e8d7c6b5a4 | ubuntu       | "/bin/bash"             | 10 minutes ago | Exited (0) 8 |

### 8. docker stop

- **Descrição:** Interrompe a execução de um container de forma graciosa (*Graceful Shutdown*).
- **Mecanismo:** Envia primeiramente o sinal **SIGTERM** para o processo principal (PID 1) do container, concedendo um período de carência (por padrão, 10 segundos) para que a aplicação feche conexões ativas, salve estados e encerre com segurança. Se o container não encerrar após o período, o Docker envia um sinal **SIGKILL** para forçar a parada.
- **Sintaxe:** `docker stop <CONTAINER_ID_OU_NOME>`

Bash



```
docker stop servidor-web
```

### 9. docker start



## INTRODUÇÃO À CONTAINERS E DOCKER

- **Descrição:** Reinicia a execução de um container previamente parado sem alterar seu estado ou suas configurações originais.
- **Sintaxe:** `docker start <CONTAINER_ID_OU_NOME>`

Bash



```
docker start servidor-web
```

### 10. docker rm

- **Descrição:** Remove permanentemente um ou mais containers parados e destrói sua camada de escrita gravável.
- **Sintaxe:** `docker rm <CONTAINER_ID_OU_NOME>`

Bash



```
docker rm servidor-web
```

💡 **Dica de Segurança:** Tentativas de remover um container ativo em execução resultarão em um erro da CLI. Para forçar a remoção de um container sem pará-lo previamente, utiliza-se a flag `-f` (`docker rm -f servidor-web`), enviando um `SIGKILL` imediato.

### 11. docker logs

- **Descrição:** Exibe os registros de auditoria e saída padrão (stdout e stderr) gerados pelas aplicações executadas dentro do container.
- **Flags Úteis:**
  - `-f` (`--follow`): Acompanha a gravação dos logs em tempo real (semelhante ao comando `tail -f`).
  - `--tail N`: Exibe apenas as últimas N linhas de log.



## INTRODUÇÃO À CONTAINERS E DOCKER

Bash



```
# Acompanha em tempo real as últimas 20 linhas de log do container  
docker logs -f --tail 20 servidor-web
```

### 12. docker inspect

- **Descrição:** Retorna uma estrutura completa de dados no formato **JSON** contendo todas as configurações de baixo nível e metadados detalhados de um objeto Docker (Container, Imagem, Rede ou Volume).
- **Informações Reveladas:** Endereço IP alocado na rede virtual, montagens de volumes, variáveis de ambiente, status de saúde, comandos de entrada e limites de cgroups.

Bash



```
docker inspect servidor-web
```

Filtrando a saída com `--format` (Exemplo para extrair apenas o Endereço IP do container):

Bash



```
docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' s
```



## INTRODUÇÃO À CONTAINERS E DOCKER

### MÓDULO 4: LABORATÓRIO E DESAFIO PRÁTICO GUIADO

Neste módulo prático, você aplicará os conceitos teóricos para criar, inspecionar, operar e gerenciar o ciclo de vida completo de um servidor Web **Nginx** containerizado.

#### Atividade Prática 1: Diagnóstico e Validação do Ambiente

##### Passo 1.1: Verificar a instalação do Docker e suas versões

Abra o seu terminal (Linux/macOS) ou o terminal do seu ambiente online (Play with Docker, Cloud Shell) e execute:

Bash

```
docker version
```

*Objetivo:* Validar se o cliente CLI e o servidor Engine estão se comunicando sem falhas de permissão.

##### Passo 1.2: Inspeção do Docker Daemon

Execute o comando de auditoria de recursos:

Bash

```
docker info
```

*Objetivo:* Identificar o driver de rede, a quantidade de recursos alocados ao Docker e quantos containers já existem no host.

#### Atividade Prática 2: Download e Gerenciamento de Imagens

##### Passo 2.1: Obter a imagem oficial do Nginx



## INTRODUÇÃO À CONTAINERS E DOCKER

Baixe a imagem do repositório oficial do Docker Hub utilizando a tag leve `alpine`:

Bash

```
docker pull nginx:alpine
```



### Passo 2.2: Listar as imagens locais disponíveis

Verifique se o download foi concluído com sucesso e analise o tamanho da imagem baixada:

Bash

```
docker images
```



*Resultado Esperado:* Uma tabela listando a imagem `nginx` com a tag `alpine` e seu Hash ID.

### Atividade Prática 3: O Desafio Prático Integrador

**Cenário do Desafio:** Você foi encarregado de implantar um servidor web Nginx em ambiente de desenvolvimento. O servidor deve:

1. Executar de forma isolada em segundo plano (*Detached*).
2. Ter o nome oficial de `meu-servidor-web`.
3. Ter a porta `80` do container exposta e mapeada para a porta `8080` do host.
4. Ser validado no navegador de internet ou via linha de comando (`curl`).
5. Ter suas propriedades de rede (Endereço IP) inspecionadas.
6. Sofrer auditoria de logs e passar pelo seu ciclo de vida (Parada, Reinício e Remoção).

### Passo 1: Executar o Container Nginx Mapeando Portas



## INTRODUÇÃO À CONTAINERS E DOCKER

Execute o comando de instanciação com os parâmetros exigidos:

Bash



```
docker run -d --name meu-servidor-web -p 8080:80 nginx:alpine
```

### Desconstrução dos Parâmetros do Comando:

- `-d`: Libera seu terminal executando o container em background.
- `--name meu-servidor-web`: Substitui o nome randômico do Docker por um identificador amigável.
- `-p 8080:80`: Cria uma regra de redirecionamento de rede. Toda requisição que chegar na porta **8080 do Host** será roteada para a porta **80 dentro do Container**.
- `nginx:alpine`: A imagem de origem utilizada como molde.

### Passo 2: Validar o Estado do Container

Verifique se o container está em execução e analise a coluna PORTS:

Bash



```
docker ps
```

### Passo 3: Acessar a Aplicação Web

Abra seu navegador de preferência e navegue para o endereço:  
`http://localhost:8080` (ou o IP da sua máquina virtual/Play with Docker).

Alternativamente, execute a validação direta via terminal em outra aba utilizando o comando `curl`:



## INTRODUÇÃO À CONTAINERS E DOCKER

Bash



```
curl http://localhost:8080
```

*Resultado Esperado:* A página HTML padrão com a mensagem **"Welcome to nginx!"**.

### Passo 4: Auditar os Logs de Acesso

Como você realizou requisições HTTP para a aplicação, o Nginx gerou logs de acesso. Visualize essas entradas no terminal:

Bash



```
docker logs meu-servidor-web
```

Observe que as requisições enviadas pelo seu navegador apareceram registradas no formato de log padrão do Nginx.

### Passo 5: Inspecionar o Endereço IP do Container

Inspeccione os detalhes estruturais do container para descobrir qual Endereço IP a rede virtual do Docker atribuiu a ele:

Bash



```
docker inspect meu-servidor-web
```

Procure pelo bloco de JSON "Networks" ou utilize o filtro para extrair diretamente a linha correspondente:



## INTRODUÇÃO À CONTAINERS E DOCKER

Bash



```
docker inspect --format='{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' meu-servidor-web
```

### Passo 6: Executar o Ciclo de Vida do Container (Parada e Reinício)

1. Parar o container em execução:

Bash



```
docker stop meu-servidor-web
```

2. Verificar se o container sumiu da listagem ativa:

Bash



```
docker ps
```

(O container não deve ser listado aqui).

3. Confirmar que o container ainda existe em estado parado:

Bash



```
docker ps -a
```

(O container será exibido com o status `Exited` ).



## INTRODUÇÃO À CONTAINERS E DOCKER

### 4. Reiniciar o container parado:

Bash



```
docker start meu-servidor-web
```

### 5. Validar a recuperação do acesso:

Bash



```
curl http://localhost:8080
```

### Passo 7: Limpeza de Recursos (Destruição do Container)

Ao encerrar o ciclo de vida de uma aplicação, o container e sua camada efêmera devem ser limpos do ambiente:

1. Pare o container novamente:

Bash



```
docker stop meu-servidor-web
```

2. Remova o container permanentemente:

Bash



```
docker rm meu-servidor-web
```



## INTRODUÇÃO À CONTAINERS E DOCKER

3. Confirme a exclusão completa:

Bash



```
docker ps -a
```

## RESUMO DOS COMANDOS EXECUTADOS NO LABORATÓRIO

Para referência rápida em estudos futuros, guarde esta tabela de comandos essenciais praticados:

Bash



```
# Diagnóstico e Informações do Ambiente
docker version
docker info

# Gerenciamento de Imagens
docker pull nginx:alpine
docker images

# Execução e Mapeamento
docker run -d --name meu-servidor-web -p 8080:80 nginx:alpine

# Inspeção e Monitoramento
docker ps
docker ps -a
docker logs meu-servidor-web
docker inspect meu-servidor-web

# Controle de Ciclo de Vida e Limpeza
docker stop meu-servidor-web
docker start meu-servidor-web
docker rm meu-servidor-web
```



**INTRODUÇÃO À CONTAINERS E DOCKER**

**REFERÊNCIAS**

**Livros e Obras de Referência**

- **COMMUNITY, Docker.** *Docker in Action*. 2. ed. Manning Publications, 2019.
- **FRAYSER, Sean.** *Docker Deep Dive: Zero to Hero*. Independently published, 2023.
- **GARTNER, P.; SHARMA, S.** *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, 2020.
- **KANE, Sean P.; MATTHIAS, Karl.** *Docker: Up & Running: Shipping Reliable Containers in Production*. 3. ed. Sebastopol: O'Reilly Media, 2023.
- **MOUAT, Adrian.** *Using Docker: Developing and Deploying Software with Containers*. Sebastopol: O'Reilly Media, 2015.
- **PULTORAK, David.** *Docker and Kubernetes for Java Developers*. Packt Publishing, 2021.
- **TURNBULL, James.** *The Docker Book: Containerization is the new virtualization*. OpenSource Press, 2014.

**Recursos Web e Documentação Oficial**

- **DOCKER.** *Docker Documentation*. Disponível em: <https://docs.docker.com/>. Acesso em: 07 out. 2026.
- **DOCKER HUB.** *Official Repositories and Image Registry*. Disponível em: <https://hub.docker.com/>. Acesso em: 07 out. 2026.
- **OPEN CONTAINER INITIATIVE (OCI).** *OCI Image Specification & Runtime Specification*. Disponível em: <https://opencontainers.org/>. Acesso em: 07 out. 2026.
- **SPRING.** *Spring Boot Reference Documentation: Efficient Container Images*. Spring / Broadcom. Disponível em: <https://docs.spring.io/spring-boot/docs/current/reference/html/container-images.html>. Acesso em: 07 out. 2026.
- **SWAGGER / SMARTBEAR.** *OpenAPI Specification*. Disponível em: <https://swagger.io/specification/>. Acesso em: 07 out. 2026.